

ID Share

Consent-based sharing of government ID documents in healthcare on Ethereum

BCS3210 Blockchains, group project
[Names of the group members]

The problem

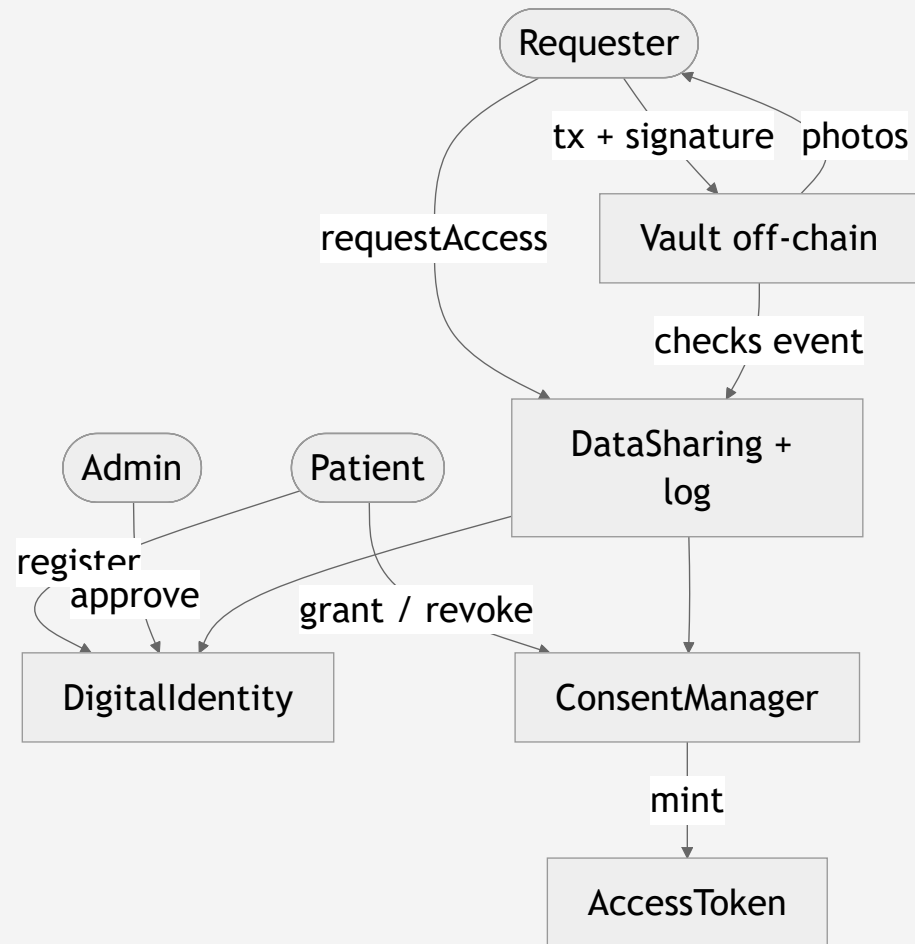
- Every clinic, lab and insurer keeps its own scan of your ID
- You don't know who has a copy or who looked at it
- You can't take a copy back
- No audit trail you can check yourself
- Every provider is a breach target: Change Healthcare 2024, **192.7 million** people affected
- Patients get nothing back for sharing

Our idea

- ID photos stay **with the patient** (off-chain vault)
- On-chain: only **hashes**, consents and the access log
- Consent: **1 to 365 days**, front / back / both, revocable any time
- **Every** access attempt logged, also the failed ones
- 10 ACT reward token for the first consent to a provider

Role	Can
Patient	register, grant / revoke consent, read log
Requester	request access, get allowed photos
Admin	approve requesters, pause

Architecture



- 4 contracts, one job each, talk through interfaces (like Lab 4)
- Only ConsentManager can mint tokens
- Python vault hands out the files
- Hardhat 3, Solidity 0.8.28, viem, web3.py

Design decision 1: failed requests must not revert

A `revert` also rolls back the log entry, so a denied attempt would leave no trace.

`requestAccess` therefore always writes the log and emits an event, and returns `true` or `false`.

```
if (!approved) reason = NotApproved;
else if (!exists) reason = NoConsent;
else if (revoked) reason = Revoked;
else if (now >= expiresAt) reason = Expired;

granted = (reason == None);
accessLogs[patient].push(...);
emit granted ? AccessGranted
             : AccessDenied(reason);
```

Design decision 2: the chain is public

- Our first version returned the file link from the contract, but anyone can read contract storage
 - So the **vault** enforces access, based on on-chain proof:
1. Requester calls `requestAccess`
 2. Sends tx hash + signature to the vault
 3. Vault checks: AccessGranted event, signer, age, not reused, consent still valid
 4. Vault sends only the allowed photos
 5. Requester checks SHA-256 against the chain

Every download has a matching entry in the on-chain audit log.

Testing

64 / 64 passing

Solidity tests with forge-std, incl. a fuzz test (256 runs)

Only generated addresses and fake hashes, no personal data.

- Boundaries (0 / 1 / 365 / 366 days), exact expiry, reward only once, denied = logged not reverted

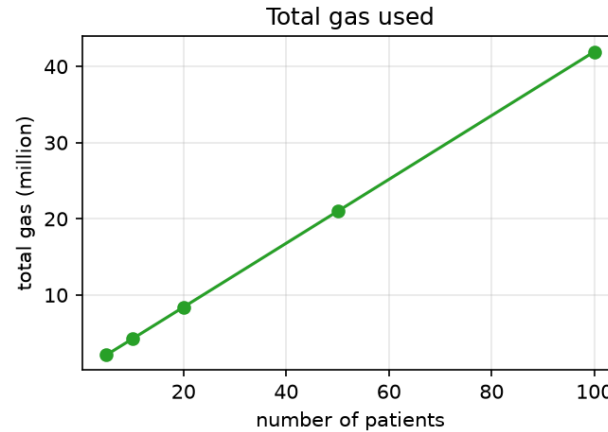
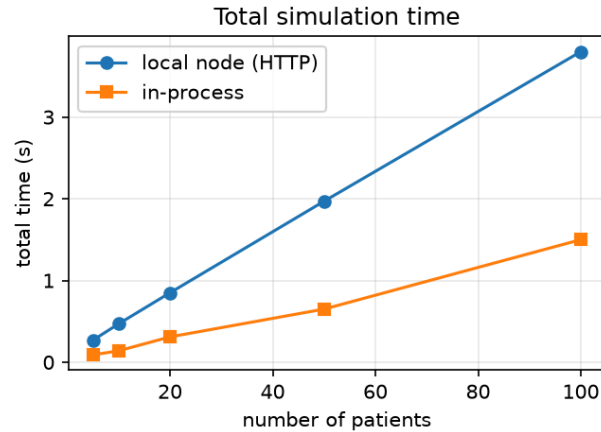
File	Tests
AccessToken.t.sol	10
DigitalIdentity.t.sol	16
ConsentManager.t.sol	21
DataSharing.t.sol	13
Integration.t.sol	4

Gas optimisation

- Timestamps as `uint64`, so a consent and a log entry each fit in **one** storage slot
- Reward flag inside the packed consent
- `immutable` contract references
- Removed a redundant counter

Operation	Before	After
requestAccess (granted)	107,429	70,108 (-35%)
grantConsent (first)	187,290	94,835 (-49%)
registerUser	167,975	145,921 (-13%)
Deployment (all)	3.14 M	3.17 M (+1%)

Simulation: 5 to 100 patients



- Gas per operation constant: no loops over users
- ~420k gas per patient lifecycle
- 100 patients: 470 tx in 3.8 s
- 8 to 10 ms confirmation on the local node
- 200 audit events read in 27 ms

Front-end (viem)

ID Share

Account

Patient 1 - 0x7099...79C8

0x70997970C51812dc3A010C7d01b50e0d17dc79C8 | 20 ACT

Patient

Requester

Admin

Audit log

My identity

Field	Value
Email hash	0xe288d12b4a68961d7d837a9803f90fe0834832581244adb5f324208993a72f83
Front photo hash	0xcbfbde808d9adce00f84b08b05f42aebef3ab306f3e77f895880476f92a9bb3b
Back photo hash	0x2058e8c5667f7d5dd251e5546446567fc89540028210eda67dfe6e6cd8c441b3
Storage reference	vault://patient1
Registered	9/30/2026, 3:16:56 PM

My consents

Requester	Scope	Valid until	State	
Doctor (0x3C44...93BC)	Front and back	10/30/2026, 2:17:04 PM	Active	Revoke
Lab (0x9965...A4dc)	Front only	10/7/2026, 3:17:16 PM	Revoked	

Give consent

Requester

Doctor (0x3C44...93BC)

What can they see

Front and back

Days (1-365)

30

Grant consent

Limitations and next steps

- Local chain: on mainnet ~12 s per block and real fees, so move to a Layer-2 rollup
- Access pattern (who asked whom, when) is public: zero-knowledge proofs could hide it
- Admin is one trusted party: use a multi-sig
- A requester can keep copies; the chain proves access but cannot force deletion
- Encrypt files for the requester, use MetaMask instead of test keys

Thank you

Questions?

Demo: `npx hardhat node`, `npm run deploy:local`, `npm run frontend`